



SECJ2013

DATA STRUCTURE AND ALGORITHM

SECTION 02

ASSIGNMENT 2:

Stack Operation

GROUP 6

MEMBER:

CHEW CHIU XIAN (A23CS0061)

HARINI A/P SANGARAN (A23CS0081)

MOHAMED ALI MOHAMED ALI (A21EC0287)

SYARIFAH DANIA BINTI SYED ABU BAKAR (A23CS0183)

LECTURER:

DR. ZURAINI BINTI ALI SHAH

Task 1: Understanding the Concept of Stack

Basic operations of a stack:

1. **Push** : add new elements on top of the stack. If the stack is full an error will occur.

```
class Stack {
private:
    int arr[MAX_SIZE];
    int top;

public:
    Stack() { top = -1; }

    void push(int value) {
        if (top == MAX_SIZE - 1) {
            cout << "Stack is full! Cannot push " << value << endl;
        } else {
            arr[++top] = value;
            cout << value << " pushed into the stack." << endl;
        }
    }
};
```

2. **Pop**: The opposite of pushing. This operation removes the element on top of the stack.

```
class Stack {
private:
    int arr[MAX_SIZE];
    int top;

public:
    Stack() { top = -1; }

    void pop() {
        if (top == -1) {
            cout << "Stack is empty! Cannot pop from an empty stack." << endl;
        } else {
            cout << arr[top--] << " popped from the stack." << endl;
        }
    }
};
```

3. **Stacktop**: Checks the element on top of the stack without removing it. Then returns it.

```
class Stack {
private:
    int arr[MAX_SIZE];
    int top;

public:
    Stack() { top = -1; }

    void stackTop() {
        if (top == -1) {
            cout << "Stack is empty." << endl;
        } else {
            cout << "Top element is: " << arr[top] << endl;
        }
    }
};
```

4. **IsEmpty**: Checks if there are elements in the stack or not. If there are no elements it will return true. It is used before Pop to make sure no error occurs.

```
class Stack {
private:
    int arr[MAX_SIZE];
    int top;

public:
    Stack() { top = -1; }

    bool isEmpty() {
        return top == -1;
    }
};
```

5. **IsFull:** Checks if the stack is full. If it is full it will return true. It is used before Push to prevent errors from occurring.

```
class Stack {
private:
    int arr[MAX_SIZE];
    int top;

public:
    Stack() { top = -1; }

    bool isFull() {
        return top == MAX_SIZE - 1;
    }
};
```

LIFO(Last In First Out):

Definition: as the name suggests this principle works with above explained operations as the last element that was put in the stack(pushes) is the first element to be taken out (popped).

Real life example:

A very good example of this operation is the undo and redo features in text editors like microsoft word, google documents and many other programs. When you want to do something the program pushes your action into the undo stack. But when you want to undo that action it is popped from the undo stack and pushed into the redo stack. And the opposite happens when you want to redo something. If you for example pressed undo too much that you deleted something you did not want to delete. All the deleted (Undid) actions you have taken will be pushed into the redo stack in order then when you press redo they will be popped from that stack based on what was pushed first it will be popped first.

Task 2: Application of Stack in a Real-World Scenario

One of the examples of real-world applications that we will be using is **Undo/Redo Functionality in Text Editors**. The app uses stack function because it follows the LIFO (Last-In-First-Out) principle, meaning the most recent change is the first to be undone, which is the natural sequence of undoing and redoing tasks.

How does the application work?

When you type or make changes in a document, each action can be stored in a stack. If you make a mistake and want to undo the action, the most recent action (the "top" of the stack) is popped off, and the document returns to its previous state.

If you then decide to redo the action, the previously undone action can be pushed back onto the stack, restoring it.

Stack operations:

- **Push:** Every action (like typing or formatting) is pushed onto the stack.
- **Pop:** An action is popped off the stack when you undo it.
- **StackTop:** Shows the most recent action on the stack.
- **IsEmpty:** Checks if there are no actions left to undo.

Task 3: Implementation in C++

Array-based stack implementation

```
#include <iostream>
```

```
using namespace std;
```

```
const int MAX = 5;
```

```
class StackArray {
```

```
    int arr[MAX];
```

```
    int top;
```

```
public:
```

```
    StackArray() { top = -1 ;}
```

```
    void push(int value) {
```

```
        if (top == MAX - 1) {
```

```
            cout << "Stack is full! Cannot add more elements.\n";
```

```
        } else {
```

```
            top++;
```

```
            arr[top] = value;
```

```
            cout << "Pushed " << value << " to the stack.\n";
```

```
        }
```

```
    }
```

```
    void pop() {
```

```
        if (top == -1) {
```

```
            cout << "Stack is empty! Nothing to remove.\n";
```

```
        } else {
```

```
            cout << "Popped " << arr[top] << " from the stack.\n";
```

```
            top--;
```

```
        }
```

```
    }
```

```
    void displayTop() {
```

```
        if (top == -1) {
```

```
            cout << "Stack is empty! No top element.\n";
```

```
        } else {
```

```
            cout << "Top element is: " << arr[top] << endl;
```

```
        }
```

```
    }
```

```
    void displayStack() {
```

```
        if (top == -1) {
```

```

        cout << "Stack is empty! Nothing to display.\n";
    } else {
        cout << "Stack elements: ";
        for (int i = 0; i <= top; i++) {
            cout << arr[i] << " ";
        }
        cout << endl;
    }
}
};

```

```

int main() {
    StackArray stack;
    int choice, value;

    do {
        cout << "Menu: " << endl;
        cout << "1. Push" << endl;
        cout << "2. Pop" << endl;
        cout << "3. Display Top" << endl;
        cout << "4. Display Stack" << endl;
        cout << "5. Exit " << endl << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
            case 1:
                cout << "Enter a value to push: ";
                cin >> value;
                stack.push(value);
                break;
            case 2:
                stack.pop();
                break;
            case 3:
                stack.displayTop();
                break;
            case 4:
                stack.displayStack();
                break;
            case 5:
                cout << "Exiting program.\n";
                break;
            default:
                cout << "Invalid choice! Please try again.\n";
        }
    } while (choice != 5);
}

```

```

    }
} while (choice != 5);

return 0;
}

```

Pointer-based stack implementation

```

#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
};

class StackPointer {
    Node* top;

public:
    StackPointer() {top = NULL;}

    void push(int value) {
        Node* newNode = new Node();
        newNode->data = value;
        newNode->next = top;
        top = newNode;
        cout << "Pushed " << value << " to the stack.\n";
    }

    void pop() {
        if (top == nullptr) {
            cout << "Stack is empty! Nothing to remove.\n";
        } else {
            cout << "Popped " << top->data << " from the stack.\n";
            Node* temp = top;
            top = top->next;
            delete temp;
        }
    }

    void displayTop() {
        if (top == NULL) {
            cout << "Stack is empty! No top element.\n";

```



```

    } else {
        cout << "Top element is: " << top->data << endl;
    }
}

void displayStack() {
    if (top == NULL) {
        cout << "Stack is empty! Nothing to display.\n";
    } else {
        cout << "Stack elements: ";
        Node* temp = top;
        while (temp != NULL) {
            cout << temp->data << " ";
            temp = temp->next;
        }
        cout << endl;
    }
}
};

```

```

int main() {
    StackPointer stack;
    int choice, value;

    do {
        cout << "Menu: " << endl;
        cout << "1. Push" << endl;
        cout << "2. Pop" << endl;
        cout << "3. Display Top" << endl;
        cout << "4. Display Stack" << endl;
        cout << "5. Exit " << endl << "Enter your choice: ";
        cin >> choice;

        switch (choice) {
            case 1:
                cout << "Enter a value to push: ";
                cin >> value;
                stack.push(value);
                break;
            case 2:
                stack.pop();
                break;
            case 3:
                stack.displayTop();

```

```

        break;
    case 4:
        stack.displayStack();
        break;
    case 5:
        cout << "Exiting program.\n";
        break;
    default:
        cout << "Invalid choice! Please try again.\n";
    }
} while (choice != 5);

return 0;
}

```

Output sample: Both array-based and pointer-based stack implementation have the same output except for the fact that array-based stack implementation is programmed to be limited to only 5 elements.

Array-based stack implementation output if

```

Menu:
1. Push
2. Pop
3. Display Top
4. Display Stack
5. Exit
Enter your choice: 1
Enter a value to push: 6
Stack is full! Cannot add more elements.

```

1) Push (1,2,3,4,5)

```
Menu:
1. Push
2. Pop
3. Display Top
4. Display Stack
5. Exit
Enter your choice: 1
Enter a value to push: 2
Pushed 2 to the stack.
```

2) Pop

```
Menu:
1. Push
2. Pop
3. Display Top
4. Display Stack
5. Exit
Enter your choice: 1
Enter a value to push: 5
Pushed 5 to the stack.
Menu:
1. Push
2. Pop
3. Display Top
4. Display Stack
5. Exit
Enter your choice: 2
Popped 5 from the stack.
```

3) Display Top after popping 5 from the stack

```
Menu:
1. Push
2. Pop
3. Display Top
4. Display Stack
5. Exit
Enter your choice: 2
Popped 5 from the stack.
Menu:
1. Push
2. Pop
3. Display Top
4. Display Stack
5. Exit
Enter your choice: 3
Top element is: 4
```

4) Display Stack

```
Menu:
1. Push
2. Pop
3. Display Top
4. Display Stack
5. Exit
Enter your choice: 4
Stack elements: 1 2 3 4
```

Task 4: Comparative Analysis/Findings

Stack can be implemented using two approaches, which are array-based implementation and pointer-based implementation. Array-based stack and pointer-based stack can be compared in terms of memory usage, ease of implementation and scalability.

Array-based stack uses static memory allocation whereas pointer-based stack uses dynamic memory allocation. Array-based stack has a fixed size. Therefore, the maximum size of the stack has to be determined in order to declare an array-based stack. If the maximum size of the array is overestimated, the memory allocated will be wasted since the memory allocated for array-based stack is fixed. Pointer-based arrays use dynamic memory allocation. Pointer-based stack does not have a fixed size. Therefore, memory will be assigned to a pointer-based stack when a new node is pushed into the stack and memory will be released when an element is popped from the stack.

Array-based stack is easier to be implemented compared to pointer-based stack. This is because array-based stack can be implemented using simple index arithmetic for operations like push and pop. Pointer-based stack is much more complex compared to array-based stack. In a pointer-based stack, a new node has to be created to push data and nodes have to be deleted when pop operation is executed. Besides that, pointers used in the stack have to be updated constantly. Hence, an array-based stack is easier to be implemented than a pointer-based stack.

Array-based stack is not scalable. As mentioned earlier, array-based stack has a fixed size. On the other hand, a pointer-based stack is scalable. The size of the pointer-based stack is not fixed. The size of the stack expands when a new node is added and the size of stack shrinks when a node is deleted. Hence, array-based stack is not scalable whereas pointer-based stack is scalable.